

AIOps Request to CDOps - Platform Context Pack

Mục đích

Tài liệu này liệt kê những thông tin AIOps cần CDOps cung cấp trước khi chốt thiết kế Self-Heal Engine và 3 contract chính: Telemetry Contract, AI API Contract, Deployment Contract.

Trong TF3 Self-Heal Engine, AIOps chịu trách nhiệm detect pattern, decide runbook/action, verify kết quả và tạo escalation context. Tuy nhiên engine chỉ có thể quyết định đúng nếu hiểu rõ hạ tầng mà CDOps dựng: cluster có service nào, tenant nằm ở đâu, signal nào có sẵn, action nào được phép, audit ghi ở đâu và rollback làm thế nào.

Nói ngắn gọn: CDOps cung cấp platform context để AIOps không thiết kế engine một cách mù. Các thông tin bên dưới sẽ được AIOps dùng trực tiếp để viết runbook, blast-radius policy, API schema, eval scenarios và safety guardrails.

Phân định trách nhiệm AIOps và CDOps

Các contract không phải là danh sách yêu cầu một chiều. AIOps là bên draft chính vì AIOps own logic của Self-Heal Engine, nhưng CDOps cần review và xác nhận tính khả thi vì CDOps là bên dựng platform, emit telemetry, execute action và cung cấp audit/verify path.

Hạng mục	AIOps chịu trách nhiệm chính	CDOps cần phản hồi/xác nhận
Telemetry Contract	Định nghĩa signal semantic, field bắt buộc, schema input cho AI Engine, signal nào dùng cho detect/decide/verify	Xác nhận có emit được không, mapping từ Prometheus/logs/traces/dataset sang schema, độ trễ, tần suất, retention, field nào cần bổ sung cho platform
AI API Contract	Định nghĩa endpoint, request/response schema, error code, idempotency, dry-run, correlation flow	Xác nhận workflow gọi API, timeout/retry/fallback, field nào cần thêm để execute action được
Deployment Contract	Định nghĩa engine cần runtime, secret, auth, audit/idempotency dependency, health/ready/metrics endpoint	Xác nhận network, security group, RBAC, audit storage, observability collector, deployment constraints
Runbook/action plan	Định nghĩa logic chọn action, blast-radius policy, verify rule, escalation rule	Xác nhận action có execute được trong sandbox không, rollback mechanism là gì, RBAC đủ không

Riêng **Telemetry Contract**: AIOps đã có baseline schema trong `Capstone-Phase-2-Code/tf-3/ai/contracts/telemetry-contract.md`. CDOps không cần tự định nghĩa lại từ đầu. Việc cần làm là review baseline này và trả lời: "Bên CDOps có emit được đúng schema này không? Nếu không, cần đổi mapping/field nào? Với các action cần

thao tác Kubernetes, CDOps có enrich được `labels.namespace` và `labels.deployment` không?"

1. Cluster Topology

AIOps cần gì

CDOps cung cấp sơ đồ hoặc bảng mô tả topology của sandbox cluster:

- namespace structure
- tenant mapping: tenant A/B nằm ở namespace nào
- service/deployment nào có trong sandbox
- dependency giữa các service
- node/pod layout nếu có

Vì sao cần

Self-Heal Engine phải biết action của nó tác động tới phần nào của hệ thống. Nếu không có topology, AIOps không thể xác định blast radius, không biết service nào critical, không biết service lỗi có dependency downstream/upstream nào, và để trả action sai phạm vi.

Ví dụ: nếu `checkout-api` lỗi, engine cần biết nó có liên quan tới `order-worker`, `queue`, `database mock` hay service khác không. Nếu restart `checkout-api` không đủ, engine có thể cần scale worker hoặc escalate thay vì retry vô hạn.

Ví dụ format CDOps có thể gửi

Tenant	Namespace	Service	Deployment	Depends on	Notes
tenant-a	tenant-a-app	checkout-api	checkout-api	order-worker, redis	public-facing API
tenant-a	tenant-a-app	order-worker	order-worker	redis	consumes queue
tenant-b	tenant-b-app	checkout-api	checkout-api	order-worker, redis	isolated from tenant-a

2. Service Catalog

AIOps cần gì

CDOps cung cấp danh sách service mẫu trong sandbox, kèm thông tin vận hành cơ bản:

- service name
- namespace
- tenant
- deployment name

- criticality
- allowed actions
- owner/platform note nếu có

Vì sao cần

Runbook của AIOps phải target đúng service/deployment thật trong sandbox. Nếu không có service catalog, engine sẽ phải hardcode hoặc đoán tên resource, dẫn tới action không chạy được trong E2E demo.

Service catalog cũng giúp AIOps phân biệt service critical và non-critical. Ví dụ action restart có thể chấp nhận với worker, nhưng với API critical thì cần dry-run, verify chặt hơn hoặc giới hạn số lần retry.

Ví dụ format CDOps có thể gửi

Service	Namespace	Tenant	Deployment	Criticality	Allowed actions
checkout-api	tenant-a-app	tenant-a	checkout-api	high	restart, rollback
order-worker	tenant-a-app	tenant-a	order-worker	medium	scale
inventory-api	tenant-b-app	tenant-b	inventory-api	high	restart

3. Allowed Action Matrix

AIOps cần gì

CDOps xác nhận rõ action nào được phép, action nào bị cấm, và giới hạn cụ thể của từng action.

Vì sao cần

Allowed action matrix là input chính cho blast-radius check. Self-Heal Engine không được tự ý thực hiện hoặc đề xuất action vượt quyền. Nếu action matrix không rõ, AIOps có thể trả action nguy hiểm hoặc action mà CDOps không thể execute.

Matrix này cũng giúp chứng minh hard requirement "zero unsafe action" trong sandbox. Khi panel hỏi vì sao action an toàn, team có thể chỉ vào policy: action chỉ được phép trong namespace tenant, tối đa N pod/deployment, không đụng IAM, không delete namespace.

Ví dụ format CDOps có thể gửi

Action	Resource	Allowed?	Limit	Notes
restart deployment	Deployment	yes	max 1 deployment/action	only tenant namespace
scale replicas	Deployment	yes	max +3 replicas, max 5 total	worker only unless approved
patch memory	Deployment	yes	max +50% current	requires pre-state

Action	Resource	Allowed?	Limit	Notes
limit			limit	snapshot
rollout undo	Deployment	yes	previous revision only	requires verify after rollback
delete pod	Pod	limited	max 1 pod, non-critical only	prefer restart deployment
delete namespace	Namespace	no	never	unsafe action
modify IAM	IAM	no	never	out of scope

4. RBAC Contract

AIOps cần gì

CDOps cung cấp quyền cụ thể của ServiceAccount hoặc identity mà engine/platform dùng để thao tác cluster:

- được `get/list/watch` resource nào
- được `patch/update/create/delete` resource nào
- namespace nào được thao tác
- resource nào tuyệt đối không được đụng
- tên ServiceAccount, Role, RoleBinding nếu đã có

Vì sao cần

AI API `/v1/decide` có thể trả về action plan hợp lệ về mặt logic, nhưng nếu RBAC không cho phép execute thì E2E sẽ fail. Ngược lại, nếu RBAC quá rộng thì không đạt yêu cầu least privilege và "zero unsafe action".

AIOps cần RBAC contract để validate action trước khi trả về. Ví dụ nếu ServiceAccount không có quyền patch deployment, engine không nên trả action `patch_memory_limit`, mà nên escalate hoặc chọn action khác.

Ví dụ format CDOps có thể gửi

Resource	Verbs	Namespace scope	Used for
Pods	get, list, watch	tenant-a-app, tenant-b-app	detect pod state, verify readiness
events	get, list, watch	tenant-a-app, tenant-b-app	detect OOMKilled, crash loop
deployments	get, list, patch	tenant-a-app, tenant-b-app	restart, scale, patch resource
deployments/scale	get, update, patch	tenant-a-app, tenant-b-app	scale worker

Resource	Verbs	Namespace scope	Used for
secrets	get, patch	selected demo namespaces only	cert rotation demo if included

5. Telemetry Schema Alignment

Trách nhiệm chính

AIOps là bên định nghĩa chính Telemetry Contract vì AI Engine cần biết chính xác signal nào dùng để detect, decide và verify. Baseline hiện tại của AIOps đã định nghĩa schema telemetry generic multi-tenant với 5 signal:

- service_error_rate
- service_latency_p95
- container_resource_usage
- application_log_event
- distributed_trace_error_event

CDOps cần review contract này và xác nhận khả năng cung cấp telemetry từ platform hoặc offline dataset. Nếu CDOps cần thêm field để routing, audit, execute action hoặc verify tốt hơn, CDOps cần đề xuất sớm trước khi contract freeze.

AIOps đã định nghĩa gì

AIOps định nghĩa:

- tên signal và ý nghĩa domain
- field bắt buộc trong payload
- timestamp format
- tenant_id requirement
- labels optional để enrich topology như namespace, deployment, pod_name, container, endpoint, trace_id
- signal nào dùng cho anomaly detection
- signal nào dùng cho post-action verify
- payload format dùng chung cho `/v1/detect.telemetry_window[]` và `/v1/verify.post_telemetry_window[]`

CDOps cần review và phản hồi gì

CDOps vui lòng phản hồi theo các câu hỏi sau:

1. Với từng signal AIOps định nghĩa, CDOps có emit được không?
2. Signal này lấy từ nguồn nào: Prometheus, Istio, K8s events, logs, traces, CSV preprocessor,

hay custom script?

3. Field nào CDOps không có sẵn và cần AIOps điều chỉnh?
4. Với các telemetry có thể dẫn tới self-heal action, CDOps có enrich được `labels.namespace` và `labels.deployment` không?
5. Tần suất emit và độ trễ thực tế là bao nhiêu?
6. Có cần đổi tên field để khớp với platform implementation không?
7. Với offline simulation mode, CDOps sẽ inject `tenant_id` UUID nào cho từng tenant/scenario?

Vì sao cần CDOps review

Telemetry là dữ liệu đầu vào cho `/v1/detect`, `/v1/decide` và `/v1/verify`. AIOps có thể định nghĩa schema đúng về mặt AI/domain, nhưng chỉ CDOps mới biết signal đó có lấy được từ hạ tầng thực tế hoặc dataset mô phỏng hay không.

Ví dụ: AIOps cần `tenant_id`, `service`, `signal_name`, `value` để detect lỗi service. Khi chuyển sang execute action Kubernetes, `/v1/decide` sẽ trả `target = deployment/<name>` và `params.namespace = <namespace>`. Nếu CDOps đã enrich `labels.namespace` và `labels.deployment` trong telemetry thì AIOps có thể đưa hai field này vào `anomaly_context` và sinh action plan chính xác hơn. Nếu hai field này không có, AIOps cần fallback bằng topology/config hoặc escalate thay vì đoán mò.

Nói cách khác: AIOps own "signal này nghĩa là gì và AI dùng để làm gì"; CDOps own "signal này lấy ở đâu, emit được không, cần field nào để platform vận hành được".

Ví dụ telemetry payload để CDOps review

Lưu ý: ví dụ dưới đây bám theo Telemetry Contract hiện tại. Các field top-level `ts`, `tenant_id`, `service`, `signal_name`, `value` là bắt buộc. Object `labels` là optional; trong đó `system`, `namespace`, `deployment` là enrichment được khuyến nghị để detect/decide/verify tốt hơn.

- field nào hiện tại đã emit được đúng contract
- field nào CDOps cần bổ sung để platform execute/audit/verify tốt hơn
- field nào không có trong dataset/platform và cần AIOps điều chỉnh lại contract

```
{
  "ts": "2026-06-25T10:30:00.123Z",
  "tenant_id": "d3b07384-d113-495f-9f58-20d18d357d75",
  "service": "order-service",
  "signal_name": "service_error_rate",
  "value": 0.085,
  "labels": {
    "system": "E-COMMERCE",
    "endpoint": "/v1/orders/checkout",
    "namespace": "production",
    "deployment": "order-service"
  }
}
```

Phân loại field trong ví dụ

Field	Trạng thái hiện tại	CDOps cần phản hồi
ts	Đã có trong Telemetry Contract	Xác nhận format RFC3339 UTC và độ chính xác timestamp
tenant_id	Đã có trong Telemetry Contract	Xác nhận tenant UUID dùng cho từng CDO/scenario
signal_name	Đã có trong AI API/Telemetry Contract	Xác nhận tên signal khi emit có khớp contract không
value	Đã có trong AI API/Telemetry Contract	Xác nhận type thực tế là number/string theo từng signal
service	Đã có trong Telemetry Contract	Xác nhận mapping từ metric/log/trace sang service name
labels.system	Required trong labels nếu labels được gửi	Xác nhận system identifier sẽ dùng
labels.namespace	Optional enrichment trong Telemetry Contract	Xác nhận có enrich được cho action-able signals không
labels.deployment	Optional enrichment trong Telemetry Contract	Xác nhận có enrich được cho action-able signals không
labels.endpoint	Optional enrichment	Xác nhận có endpoint/operation đầy đủ không

Ví dụ bảng review từ CDOps

Signal AIOps định nghĩa	CDOps emit được?	Source	Mapping/transform	Field cần bổ sung	Notes
service_error_rate	yes/no	Prometheus/Istio/CSV	error requests / total requests	namespace, deployment nếu action-able	dùng cho detect + verify
service_latency_p95	yes/no	Prometheus/Istio/CSV	p95 latency window	endpoint, namespace, deployment	dùng cho service stuck
container_resource_usage	yes/no	cAdvisor/K8s metrics/CSV	memory/CPU usage	pod_name, container	dùng cho memory leak/OOM prevention
application_log_event	yes/no	Loki/CloudWatch/log CSV	filter ERROR/stack trace	pod_name, level	dùng cho context bundle
distributed_trace_error_event	yes/no	Jaeger/X-Ray/trace CSV	failed span/status	trace_id, span_id, operation	cần confirm trace field

Ví dụ yêu cầu bổ sung hợp lý từ CDOps

Nếu CDOps cần thêm field, nên ghi rõ lý do:

Field đề xuất	Vì sao CDOps cần	AIOps impact
labels.namespace	cần xác định namespace khi sinh K8s action	dùng trong anomaly_context và /v1/decide.action_plan[].params.namespace
labels.deployment	cần map service sang Deployment target	dùng trong anomaly_context và /v1/decide.action_plan[].target
pre_state_snapshot_id	link telemetry với snapshot trước action	dùng audit và rollback

6. Incident Injection Plan

AIOps cần gì

CDOps mô tả cách tạo lỗi trong sandbox, expected signal sau khi inject, và cách cleanup.

Vì sao cần

Hard requirement yêu cầu scenario simulation tối thiểu 4h và ít nhất 10 scenarios. AIOps cần incident injection plan để xây eval set, expected outcome và auto-resolve report.

Nếu injection không ổn định hoặc không tạo signal đúng schema, AIOps sẽ không test được engine. Vì vậy mỗi scenario cần có cách inject rõ ràng, signal kỳ vọng và tiêu chí pass/fail.

Ví dụ format CDOps có thể gửi

Scenario	How injected	Expected observable signal	Cleanup
Pod OOM	set memory limit thấp hoặc tạo load cao	OOMKilled event, restart_count tăng	restore memory limit
Service stuck	app sleep/hang hoặc readiness fail	readiness fail, latency tăng, available replicas giảm	restart deployment
Queue backlog	push nhiều jobs vào queue mock	queue_depth tăng, worker lag tăng	drain queue
Cert expiring	fake cert metadata hoặc secret annotation	cert_expiry_days thấp	reset annotation
Unsafe action case	target namespace forbidden	blast-radius denied	no action

7. Pre-state Snapshot Format

AIOps cần gì

CDOps xác nhận trước khi action thì snapshot trạng thái nào sẽ được lấy, ai lấy snapshot, và format lưu vào audit ra sao.

Các thông tin nên có:

- deployment YAML/spec
- replica count
- image tag
- resource limits/requests
- config version hoặc secret version nếu liên quan
- current health metrics

Vì sao cần

Pre-state snapshot phục vụ hai mục tiêu: rollback và audit. Nếu action làm hệ thống tệ hơn, engine/CDOps cần biết trạng thái trước đó để khôi phục. Nếu panel hỏi engine đã làm gì và có thể đảo ngược không, snapshot là bằng chứng rõ nhất.

Ví dụ action `patch_memory_limit` phải lưu memory limit cũ. Action `scale worker` phải lưu replica count cũ. Action `rollback deployment` phải lưu image/revision trước khi rollback.

Ví dụ snapshot

```
{
  "snapshot_id": "snap-20260624-001",
  "tenant_id": "tenant-a",
  "namespace": "tenant-a-app",
  "resource_kind": "Deployment",
  "resource_name": "order-worker",
  "replicas": 2,
  "image": "order-worker:v1.4.2",
  "resources": {
    "requests": {"cpu": "200m", "memory": "256Mi"},
    "limits": {"cpu": "500m", "memory": "512Mi"}
  },
  "captured_at": "2026-06-24T10:30:00Z"
}
```

8. Audit Storage Interface

AIOps cần gì

CDOps cung cấp interface để ghi và query audit log:

- ghi audit bằng API nào hoặc SDK nào
- schema log bắt buộc

- S3 bucket/table name hoặc endpoint
- Object Lock/append-only đã bật chưa
- retention bao lâu
- query bằng Athena/UI/log viewer như thế nào

Vì sao cần

Audit là hard requirement của TF3. AIOps có thể tạo audit event, nhưng cần CDOps cung cấp nơi lưu trữ tamper-evident và cách query khi demo.

Audit log phải cho thấy toàn bộ chuỗi quyết định: detect input, matched pattern, action plan, safety check, pre-state snapshot, execution result, verify result, rollback/escalation nếu có.

Ví dụ audit event AIOps muốn ghi

```
{
  "audit_id": "audit-001",
  "correlation_id": "inc-oom-001",
  "idempotency_key": "tenant-a-oom-checkout-001",
  "tenant_id": "tenant-a",
  "event_type": "decision",
  "matched_pattern": "pod_oom_killed",
  "dry_run_mode": false,
  "action_plan": {
    "action": "restart_deployment",
    "namespace": "tenant-a-app",
    "deployment": "checkout-api"
  },
  "safety": {
    "blast_radius_allowed": true,
    "max_pods_affected": 2,
    "circuit_breaker_open": false
  },
  "created_at": "2026-06-24T10:30:00Z"
}
```

9. Verification Metric Source

AIOps cần gì

CDOps xác nhận sau action thì dữ liệu gửi vào `/v1/verify` được lấy từ đâu:

- CDOps gửi post-state metrics vào request body
- engine tự query Prometheus
- engine gọi một CDOps metrics endpoint
- CDOps workflow tự query metric rồi gọi `/v1/verify`

Theo AI API Contract hiện tại, CDOps sẽ gọi `/v1/verify` với:

- headers: X-Tenant-Id, Idempotency-Key
- body: correlation_id, action_executed, post_telemetry_window

Trong đó `post_telemetry_window` phải dùng đúng form telemetry đã định nghĩa trong Telemetry Contract.

Vì sao cần

"Auto-resolved" không nên được hiểu là "command chạy xong". Với Self-Heal, auto-resolved nghĩa là metric hoặc trạng thái hệ thống đã quay về ngưỡng bình thường trong một verify window.

Nếu không chốt verification source, AIOps không thể định nghĩa success/fail rõ ràng. Ví dụ restart deployment thành công nhưng error rate vẫn cao thì không được tính là resolved; engine phải rollback hoặc escalate.

CDOps cần phản hồi gì

CDOps vui lòng xác nhận:

1. Sau khi execute action, CDOps sẽ lấy `post_telemetry_window` từ nguồn nào?
2. Verify window dài bao lâu? Ví dụ 2 phút, 5 phút hay theo từng scenario?
3. CDOps có gửi đủ telemetry theo Telemetry Contract không?
4. Với offline simulation mode, mốc thời gian "sau action" được map sang dòng/cửa sổ nào trong dữ liệu lịch sử?
5. Nếu action execute fail, CDOps sẽ vẫn gọi `/v1/verify` với `status = FAILED` hay chuyển thẳng escalation/fallback?

Ví dụ verify request theo AI API Contract

```
{
  "correlation_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "action_executed": {
    "action": "RESTART_DEPLOYMENT",
    "target": "deployment/order-service",
    "status": "COMPLETED",
    "execution_time_seconds": 45
  },
  "post_telemetry_window": [
    {
      "ts": "2026-06-25T10:02:00.000Z",
      "tenant_id": "d3b07384-d113-495f-9f58-20d18d357d75",
      "service": "order-service",
      "signal_name": "service_error_rate",
      "value": 0.0,
      "labels": {
        "system": "E-COMMERCE",
        "namespace": "production",
        "deployment": "order-service"
      }
    },
    {
      "ts": "2026-06-25T10:02:05.000Z",
      "tenant_id": "d3b07384-d113-495f-9f58-20d18d357d75",
      "service": "order-service",
      "signal_name": "service_latency_p95",
      "value": 120.0,
      "labels": {
        "system": "E-COMMERCE",
```

```
    "namespace": "production",  
    "deployment": "order-service"  
  }  
}  
]  
}
```

Headers tương ứng:

```
POST /v1/verify  
X-Tenant-Id: d3b07384-d113-495f-9f58-20d18d357d75  
Idempotency-Key: 8f7c6d5e-1234-4abc-9def-111122223333  
Authorization: IAM SigV4
```

Suggested Response Checklist for CDOps

CDOps có thể phản hồi theo checklist sau:

- ☐ Cluster topology diagram/table
- ☐ Service catalog
- ☐ Tenant to namespace mapping
- ☐ Allowed action matrix
- ☐ RBAC permissions/ServiceAccount details
- ☐ Telemetry schema and sample payloads
- ☐ Alert source format and sample alerts
- ☐ Incident injection plan
- ☐ Pre-state snapshot format
- ☐ Audit storage interface
- ☐ Verification metric source
- ☐ Rollback mechanism per action
- ☐ Network/auth details for calling AI engine

Priority Nếu Không Đủ Thời Gian

Nếu không kịp cung cấp toàn bộ, AIOps cần CDOps ưu tiên 5 phần sau trước:

1. Cluster topology
2. Service catalog + tenant mapping
3. Telemetry schema + sample alert payloads
4. Allowed action matrix + RBAC
5. Incident injection scenarios

Có 5 phần này, AIOps có thể chốt phần lớn runbook, API schema, safety policy và eval scenarios cho Self-Heal Engine.

